Prof. Dr. Thomas Schultz
Jonathan Lennartz (jlen@uni-bonn.de)
Annika Mikliss (amikliss@uni-bonn.de)

University of Bonn
B-IT/Computer Science II
May 23, 2025

Summer term 2025

Visual Data Analysis

Assignment Sheet 8

Solution has to be submitted via eCampus by May 30, 2025, 10:00 a.m.

If you have questions concerning the exercises, please use the forum on eCampus.

- Please work on this exercise in **small groups** of 3 students. Submit each solution only once, but clearly indicate who contributed to it by forming a team in eCampus. Remember that all team members have to be able to explain all answers.
- Please submit your answers in PDF format, and your scripts as *.py/*.ipynb files. If you are using Jupyter notebook, please also export your scripts and results as PDF.

Exercise 1 (Mapping Wind Power Generation, *20 Points*)

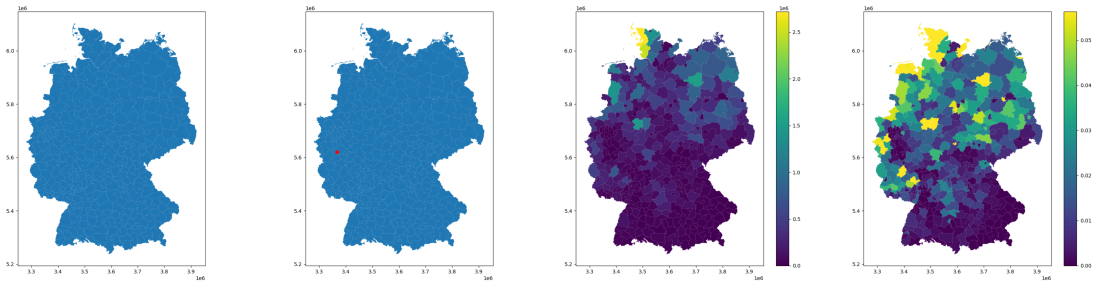


Figure 1: A screenshot of the maps you are asked to create in this task. From left to right: administrative areas of Germany, Bonn marked, net wind power generation by district, normalized by area.

This task introduces you to the `geopandas` package, which makes it quite easy to generate certain types of geospatial data visualizations. The Getting Started tutorial might be helpful.

For this exercise, we provide geospatial data of all Landkreise (administrative regions) in Germany in shapefile format. In particular, you will find five files with the following extensions:

- `.shp`: contains the feature geometry itself
- `.shx`: contains a positional index of the feature geometry to allow seeking forwards and backwards quickly
- `.dbf`: contains attributes for each shape, in dBase IV format
- `.cpg`: specifies the character encoding that is used within the `.dbf` file
- `.prj`: projection description, using a well-known text representation of coordinate reference systems

If you read in the `.shp` file with geopandas, the remaining files are read automatically and will yield the polygons as well as some additional information on the Landkreise. Use this data to solve the following tasks:

a) Create a plot that shows the administrative areas of Germany. (2P)

b) Find Bonn in the dataset and plot the polygon for Bonn. (2P)

c) Add a marker at the location of Bonn to your administrative map of Germany. (2P)
Hint: You somehow need to turn the polygon into a single point.

You will now create your own power generation map: it should use colors to show how much power is generated within each administrative district. To do this, you need to combine the shapefile dataset with the power generation dataset `power_generation.csv`, which is also in the `districts` folder (Source: https://www.marktstammdatenregister.de/MaStR).
The power generation file contains data about each single power plant in Germany.

d) Please calculate the accumulated power generation for each administrative district and plant type. (2P)

e) Both datasets use the same IDs for the administrative districts. Merge the two datasets so that the entries are correctly matched by administrative districts for the plant type `wind`. (4P)

f) Plot a map that encodes the generated wind power for each administrative district. Include a legend that tells the reader how to interpret the colors. (2P)

g) Currently, the colormap is not very meaningful because there are two districts producing much more power than all other districts. Recreate the plot with optimized utilization of the colorscale range by clipping the upper bound. (2P)

h) Employ `geopandas` to compute the size of each administrative district in Germany, and create a plot of the generated wind power, normalized by area. (3P)

i) What pattern do you observe in the distribution of wind power generation across Germany? (1P)

Exercise 2 (Brushing and Linking Between Dynamic Views in Dash, *20 Points*)

Continuing the introduction to Dash from the previous sheet, you will now learn how to dynamically remove views in Dash, and how to link user selections between them. This requires the use of so-called Pattern-Matching Callbacks, which are used to receive callbacks from a dynamic number of components.

You should find the Dash tutorial, as well as the documentation chapter on Pattern Matching Callbacks, useful for solving this exercise (https://dash.plotly.com/). We will make an example solution for the previous exercise (on which we build here) available after the respective submission deadline.

a) Implement a checkbutton next to each plot. Use the ALL selector (see Pattern-Matching Callbacks doc) to implement a button which deletes all checked plots. (7P)
Hints:

- Instead of appending single plots to the children list of your parent Div container, you may want to append whole Div containers that contain both the plot and the check button.
- To use the ALL selector in the deletion part, you need to be able to refer to several components at once. This requires that you set up their component IDs in a specific way: each ID should be a dictionary, containing a ”type” entry that is the same for all components of the same type, as well as an ”index” that is unique to each component of the same type.



Figure 2: A screenshot of the interactive visualization you are asked to implement in this exercise.

- You have to use "dash.callback_context" to check which inputs triggered the callback (see the Pattern-Matching Callbacks doc for details), and then use conditional statements to react accordingly.

b) Use the MATCH selector to implement another button, which shows the minimum and maximum values in x and y direction for the corresponding plot. (7P)

Hint: You can write a separate callback from the previous one here!

c) Implement brushing and linking: a selection made in any of the plots should be highlighted in all other plots. (6P)

Hints:

- This needs to be implemented in your big previous callback again.
- Use your dcc.Graph's selectedData property as an input.

Exercise 3 (Bilinear and Barycentric Interpolation, *6 Points*)

You are given points A (0,0), B (0,1), C (1,1) and D (1,0) on a plane with the following values: A=1, B=2, C=3, D=4. Calculate the value of E=(1/4,1/2) by bilinear interpolation and barycentric interpolation with respect to the two possible triangulations in the 2nd and 3rd drawing in Figure 3.

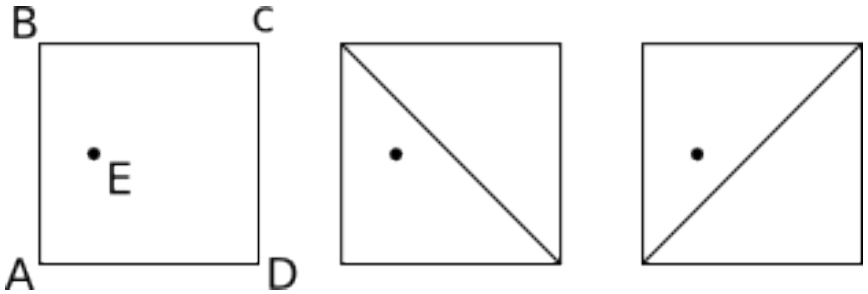


Figure 3: A grid cell ABCD and a point E at which to interpolate.

Exercise 4 (Types of Structured Grids, *4 Points*)

a) All uniform grids can also be stored as rectilinear grids. Under which condition can we store a rectilinear grid as a uniform grid? (2P)

b) Can curvilinear grids be used to represent anisotropic voxels? Justify your answer. (2P)

Good Luck!